

# 1 Secure Cloud Application Architecture

## 1.1 Architecture Overview

The architecture consists of the following major components:

**Client:** The client represents the external user interacting with the system. Communications from the client to the cloud infrastructure may be through secure HTTPS connections or insecure connections(which will be dealt with at next trust boundary).

**API Gateway:** The API Gateway serves as the entry point for all incoming requests. It validates requests before forwarding them to the application layer. This component performs the following security functions:  
Only valid requests are forwarded to the application server.

**Application Server:** The application server handles business logic and processes validated requests received from the API Gateway. The server performs authentication, authorization, and input validation before accessing storage services.

**Storage Servers:** The storage layer maintains application data. It is accessible only from the application server and is deployed in a restricted internal network zone.

**Security Logging Module:** The API Gateway forwards security-related events to the logging and monitoring module. This component records system activities and detects abnormal behavior such as repeated login failures or excessive request rates.

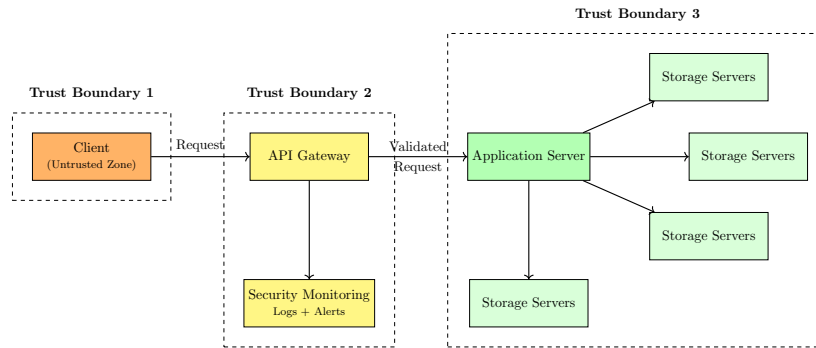


Figure 1: Secure Cloud Architecture with Trust Boundaries

## 1.2 Data Flow Description

The secure data flow within the architecture is as follows:

1. The client sends a request to the API Gateway.
2. The API Gateway validates the request.
3. If the request is valid, it is forwarded to the Application Server.
4. The Application Server processes the request and interacts with the Storage Servers if required.
5. Security-related events are logged and monitored continuously.

## 1.3 Trust Boundaries

The architecture defines clear trust boundaries:

- Between the Client and API Gateway (untrusted external zone).
- Between the API Gateway and Application Server (internal trusted zone).
- Between the Application Server and Storage Servers (restricted data zone).

This separation ensures that even if one component is compromised, the impact on other layers is minimized.

## 1.4 Security Enforcement Points

Security controls are distributed across the architecture:

- API Gateway: Request filtering and rate limiting.
- Application Server: Authentication and role-based access control.
- Storage Servers: Access restriction and data protection.
- Logging Module: Continuous monitoring and threat detection.

This layered approach improves system resilience and supports automated threat detection and mitigation.

## 2 Threat Planning and Simulation

This section describes the security threats selected for simulation. Each threat includes its identification method and the corresponding automated mitigation strategy implemented in the system.

### 2.1 Brute Force Login Attack

**Identification:** This attack is identified by monitoring repeated failed login attempts from the same user account or IP address within a short time window. If the number of failed attempts exceeds a predefined threshold, the activity is classified as a brute force attempt.

**Solution:** The system automatically locks the user account temporarily and logs the event in the security monitoring module. Additionally, rate limiting is applied at the API Gateway to restrict excessive login requests.

### 2.2 Denial of Service (DoS) Attack

**Identification:** The attack is detected when an unusually high number of requests are received from a single IP address or multiple sources within a short period. Request rate monitoring at the API Gateway identifies traffic anomalies.

**Solution:** The API Gateway enforces rate limiting and temporarily blocks offending IP addresses. Suspicious traffic patterns are logged for further analysis.

### 2.3 Unauthorized Access Attempt

**Identification:** This threat is identified when a user attempts to access restricted resources without sufficient privileges. The application server detects such attempts during role-based access control verification.

**Solution:** Access is denied with an appropriate error response. The attempt is recorded in security logs, and repeated violations may trigger additional monitoring.

### 2.4 Invalid / Tampered JWT Token

**Identification:** The threat is detected when a received JSON Web Token fails signature verification or contains altered payload information. Token validation is performed at the application server before processing requests.

**Solution:** The request is rejected immediately, and the event is logged. Invalid tokens are not processed further, preventing unauthorized access.

## 2.5 Replay Attack

**Identification:** A replay attack is identified when an expired token or previously used authentication token is reused. Token expiration time and unique token identifiers are verified during authentication.

**Solution:** Expired or reused tokens are rejected. The system enforces short token validity periods and may implement token blacklisting to prevent reuse.

## 2.6 Insecure Direct Object Reference (IDOR) Attack

**Identification:** This attack is detected when a user attempts to access data objects by directly modifying object identifiers (e.g., user IDs) in API requests. Authorization checks verify whether the requesting user is permitted to access the specified resource.

**Solution:** The system enforces strict authorization checks before granting access to any object. Requests that fail authorization are denied and logged for security auditing.